State space search

Algorithm and strategies

Stepas Toliautas

09/22/26

Searchable environments

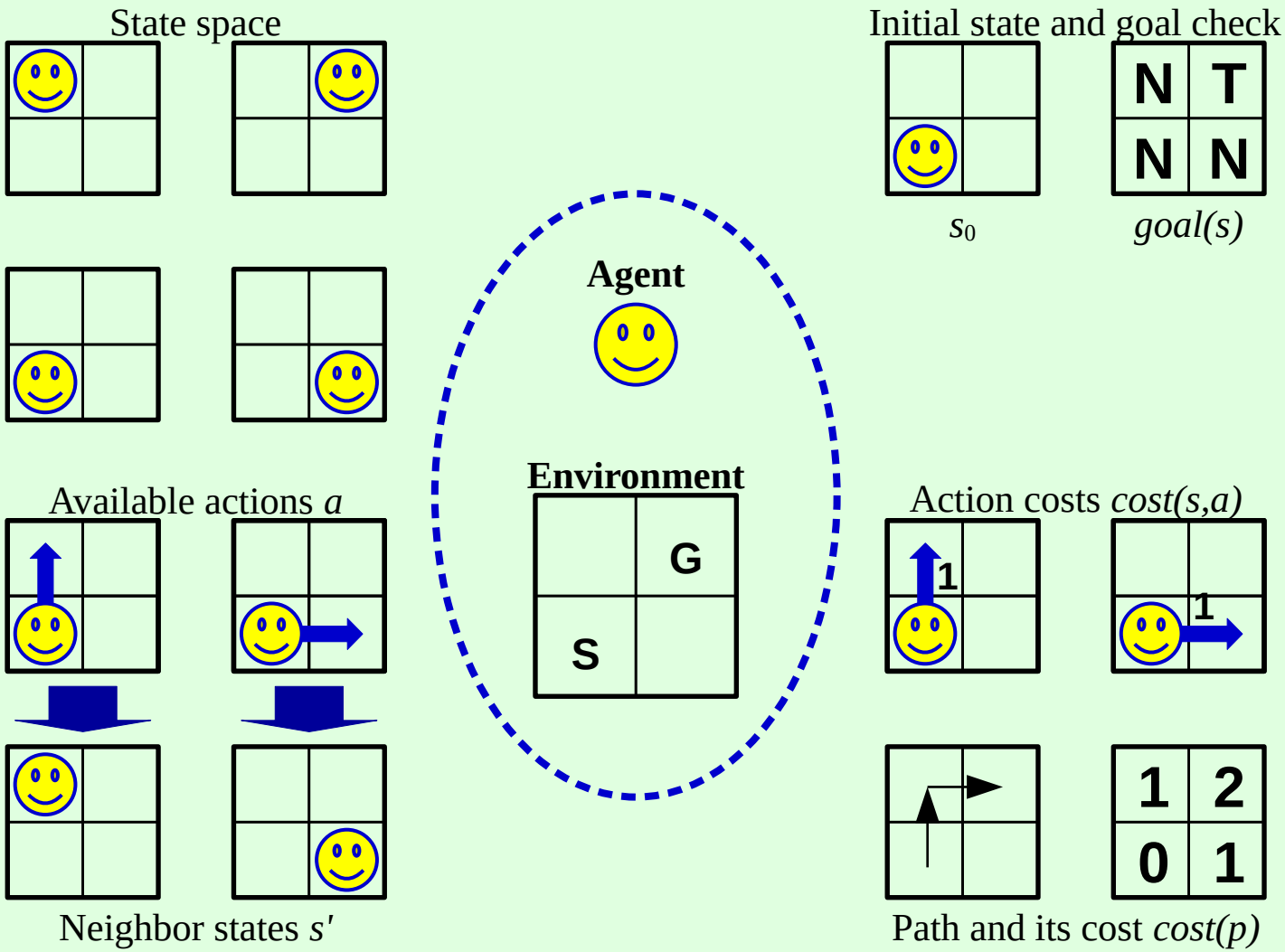
- Fully observable
  - State structure is known
- Discrete and atomic (no sub-states)
  - A common approximation even for complex systems
- Deterministic
- Static

- Solution is a fixed *sequence of steps*
  - Process of solving does not require further observation
  - Suitable problem formulation is almost equivalent to solution

Search: notation

- State space $\{s\}$
  - Initial state s_0
- State traversal rules:
  - Possible moves, *actions(s)* => $\{a\}$ and their results, *move(s,a)* => *s'*
  - Successor (neighbor) list, *successors(s)* => $\{(a,s')\}$
- Goal:
  - Goal state s_g
  - Goal check, *goal(s) = true/false*
- Additional information
  - Path to goal, $p = [s_0, a_0, s_1, a_1, ..., s_g]$ (or two separate lists?)
  - Cost function, *cost(s,a)* or *cost(p)*
  - Heuristic function *h(s)*

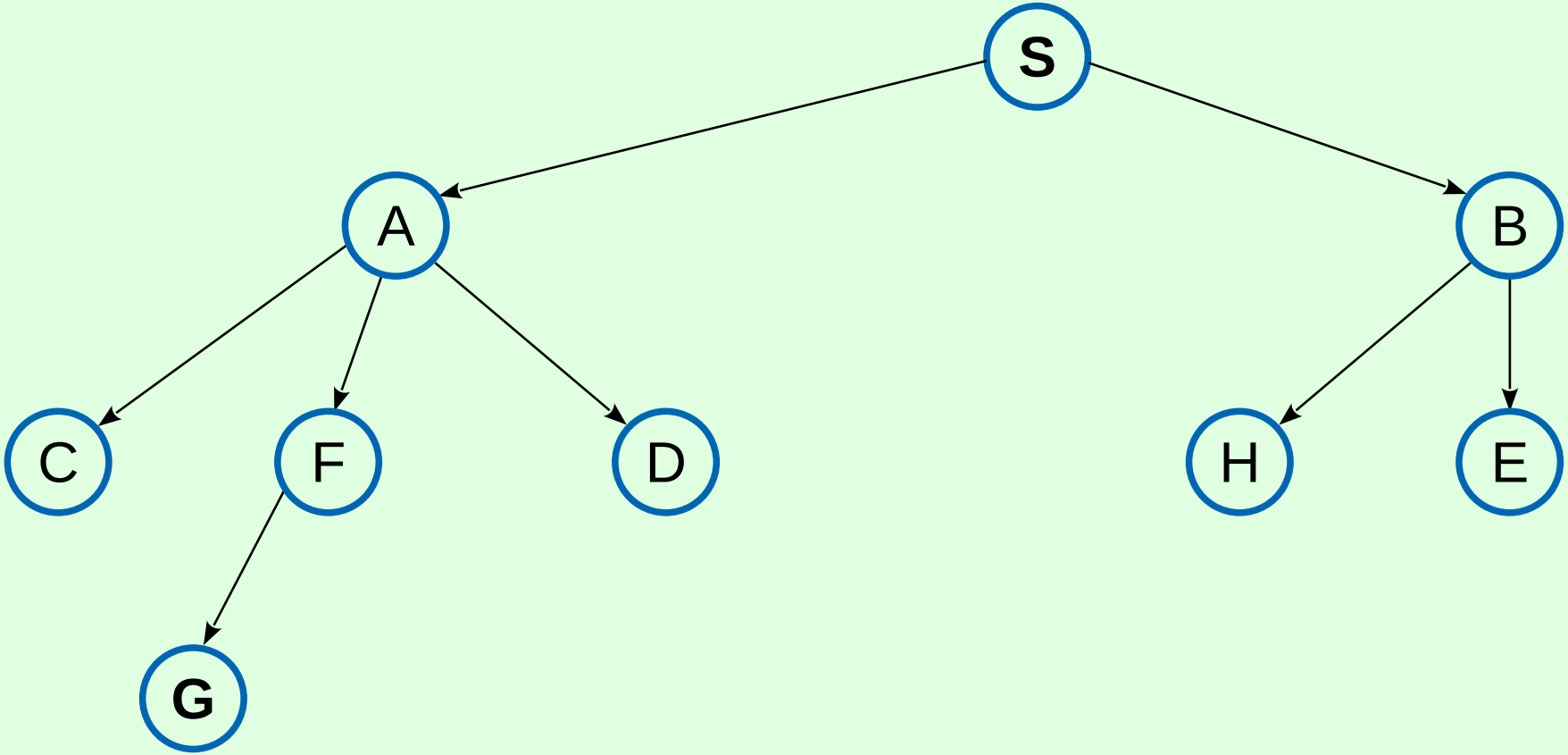
Model of searchable space



Uninformed search

- Available information:
  - Initial state
  - Traversal scheme
  - Goal check function
- Solving progresses by covering state space
  - New states and paths are „discovered“
  - Traversal schemes allow for infinite as well as finite spaces
  - Goal check serves as stopping condition
- Search works by constructing state tree* in memory

Tree search

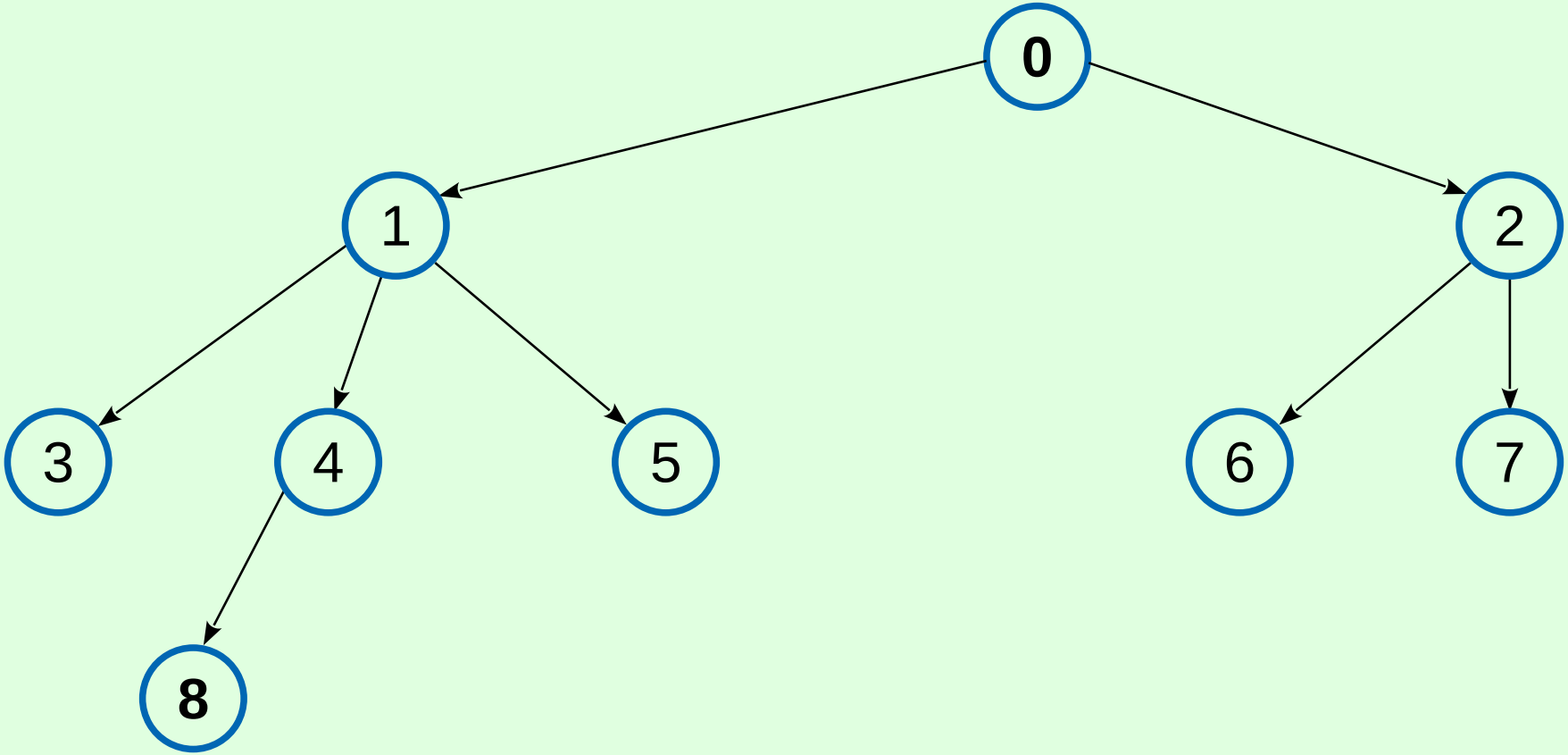


Algorithm of tree search

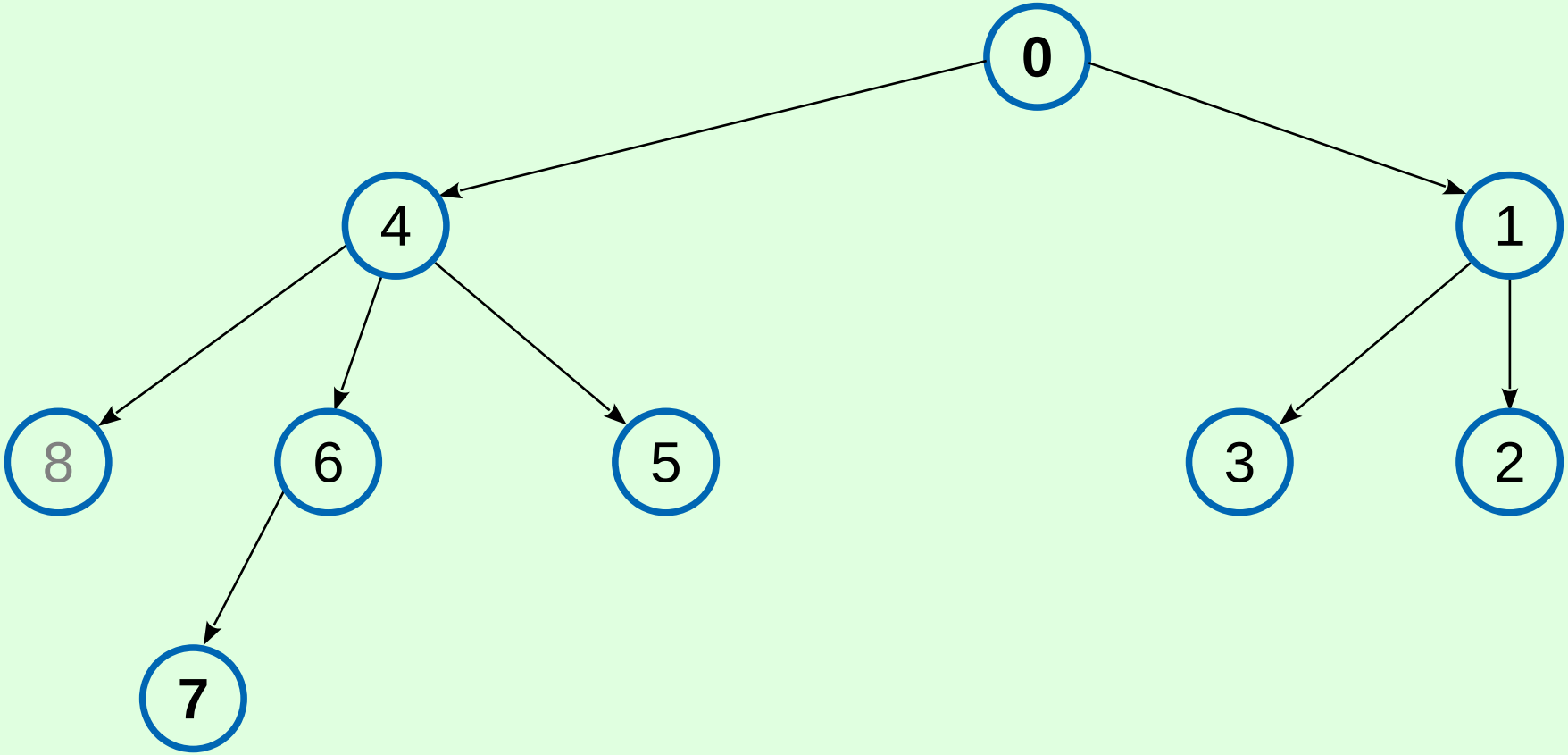
```
def treesearch (model):
    (start, goal, successors) = model
    if goal(start): return [start]
    frontier = [ [start] ]
    while frontier:
        path = get_path(frontier)
        state = path[-1]
        for (action, new_state) in successors(state):
            new_path = path + [action, new_state]
            if goal(new_state): return new_path
            frontier.append(new_path)
    return []
```

- *frontier* – list of paths ending in new (unexplored) states
- *get_path* – path selection routine (search ***strategy***)
  - Going *down* (select newest path) vs. *across* (oldest path)
  - *depth-first search* (DFS) and *breadth-first search* (BFS)
  - ***Note*** that successor function also influences path ordering!

Breadth-first search (BFS)



Depth-first search (DFS)



Evaluation of search strategies

- Completeness
  - Is the *finite* solution invariably found?
- Optimality
  - Is the *best* solution always found?
- Time and memory algorithmic complexity
  - How are resources used during search?

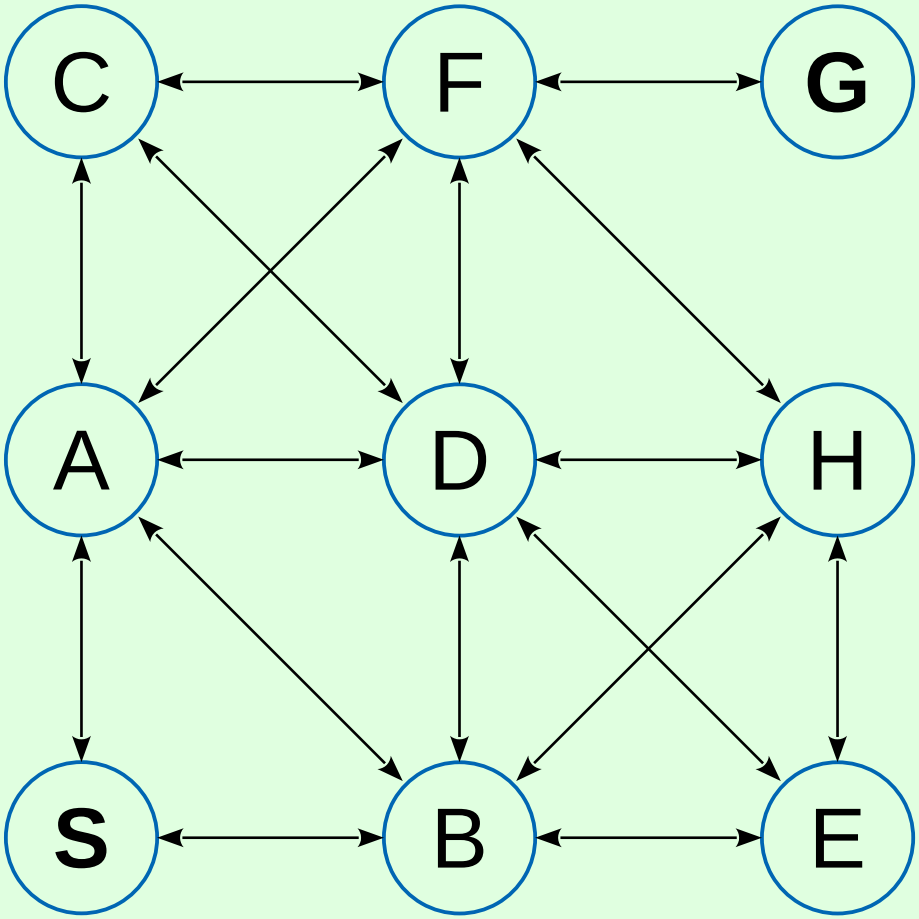
- Parameters:
  - *b* – branching factor (neighbor count)
  - *m* – tree depth (can be infinite!)
  - *d* – solution depth (path length)

Evaluation: BFS and DFS

- BFS – complete and optimal algorithm
  - Both time and memory costs are $O(b^d)$
- DFS – ***in***complete and ***non-***optimal algorithm
  - Finite solution in infinite space is not necessarily found
  - First located solution is not guaranteed to be the best
  - Time complexity is $O(b^m)$, memory – ***O(bm)***

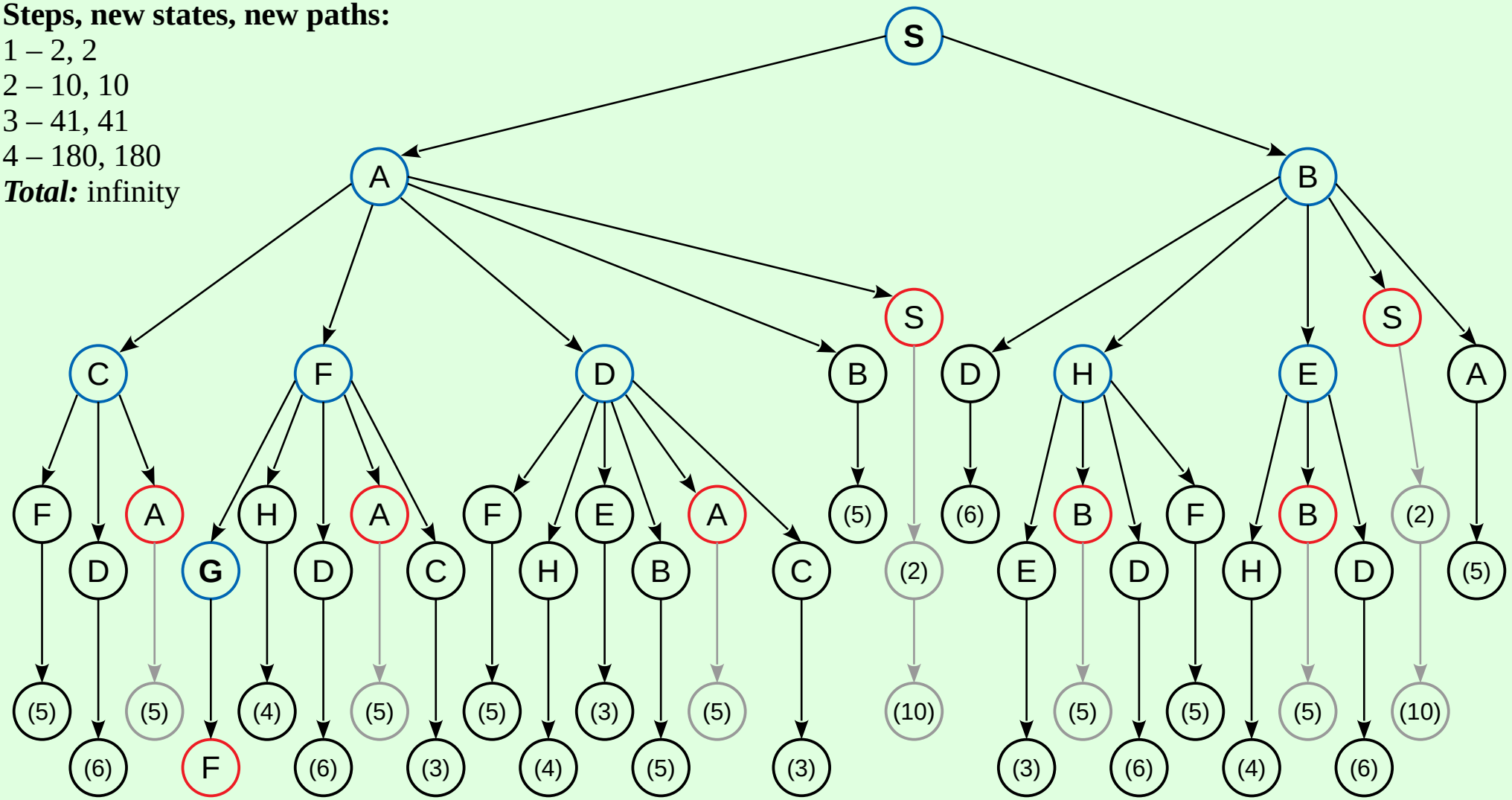
- DFS drawbacks are avoided using *iterative* search
  - *iterative deepening search* (IDS)
  - Limited depth search is performed at increasing depth levels
  - Complete and optimal algorithm*
  - Time costs are $O(b^d + \varepsilon)$, memory – $O(bd)$
  - ε denotes time spent for first d–1 iterations

Repeated states: graph search



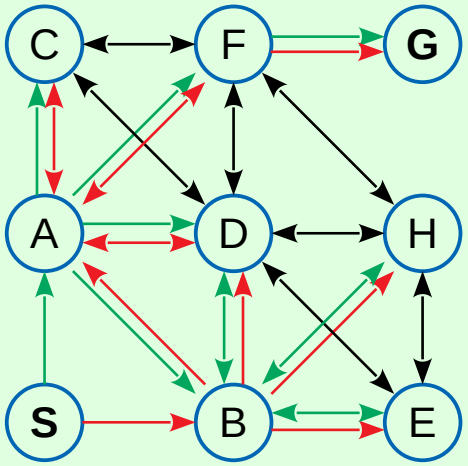
Explosion of search tree

Steps, new states, new paths:
1 – 2, 2
2 – 10, 10
3 – 41, 41
4 – 180, 180
Total: infinity



Loop elimination

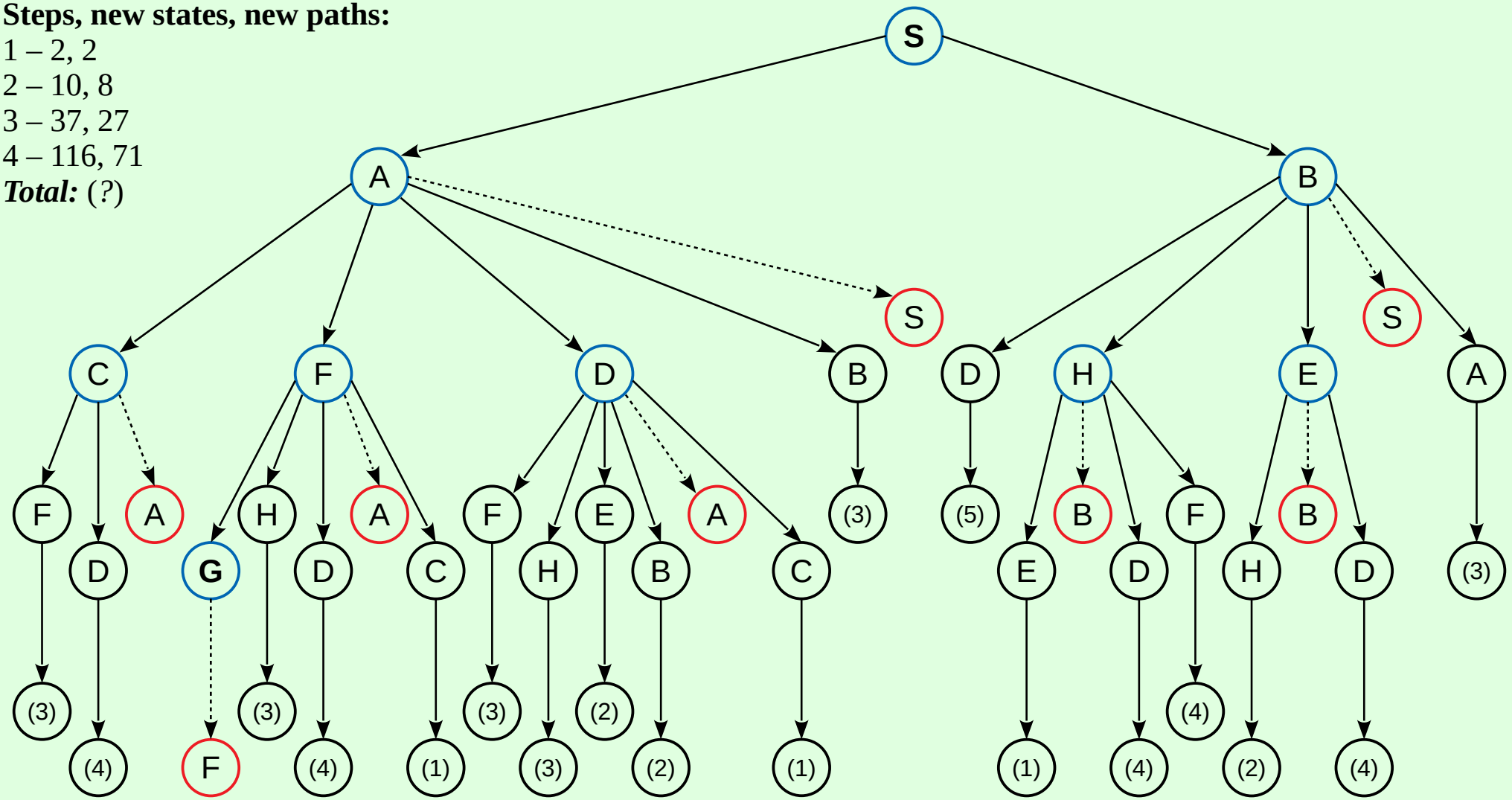
```
def treesearchplus (model, depth):
    (start, goal, successors) = model
    if goal(start): return [start]
    frontier = [ [start] ]
    while frontier:
        path = get_path(frontier)
        if path_len(path) >= depth: continue
        state = path[-1]
        for (action, new_state) in successors(state):
            if new_state not in path:
                new_path = path + [action, new_state]
                if goal(new_state): return new_path
                frontier.append(new_path)
    return []
```



- Visited states are *ignored*
  - Each single path contains only unique states
  - State duplication remains in different paths within frontier

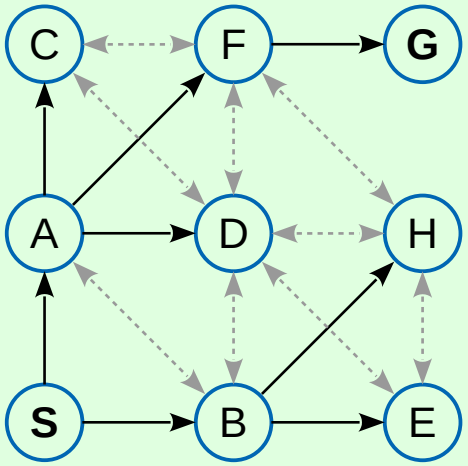
Loop-free search tree

Steps, new states, new paths:
1 – 2, 2
2 – 10, 8
3 – 37, 27
4 – 116, 71
Total: (*?*)



State caching

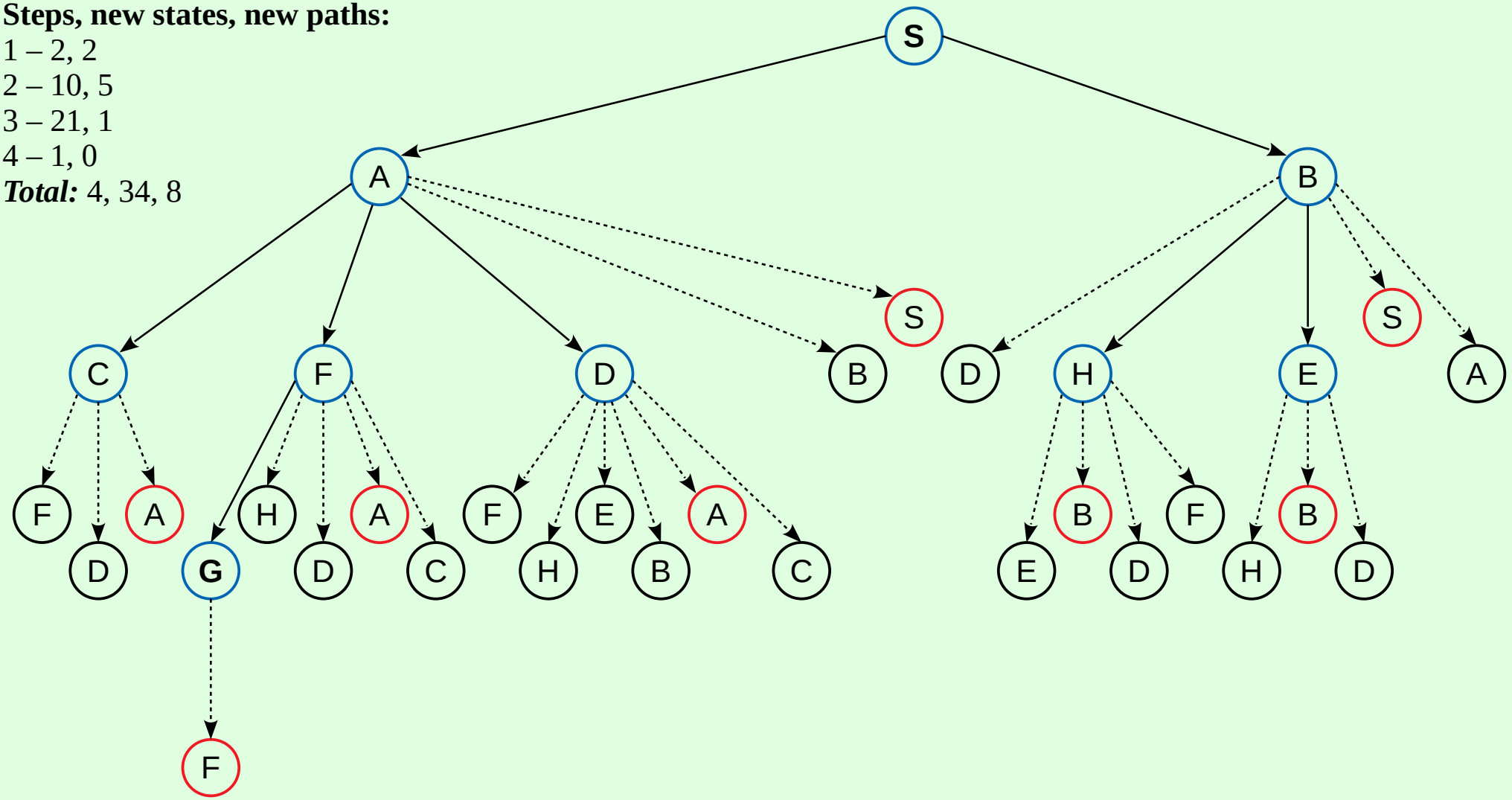
```
def graphsearch (model, depth):
    (start, goal, successors) = model
    if goal(start): return [start]
    frontier = [ [start] ]
    explored = set([ start ])
    while frontier:
        path = get_path(frontier)
        if path_len(path) >= depth: continue
        state = path[-1]
        for (action, new_state) in successors(state):
            if new_state not in explored:
                new_path = path + [action, new_state]
                if goal(new_state): return new_path
                frontier.append(new_path)
                explored.add(new_state)
    return []
```



- Visited states are *stored* and checked against
  - Each state is *expanded* only once
  - In case of DFS state depth is also recorded

Search tree with caching

Steps, new states, new paths:
1 – 2, 2
2 – 10, 5
3 – 21, 1
4 – 1, 0
Total: 4, 34, 8



Path cost

- BFS and DFS use *step count* as path cost
  - Best solution is *shortest* solution
- Search can be expanded to work with varying costs
  - *cost(p)* must be strictly-increasing function
  - Frontier must be **sorted** by path cost
  - Only **lowest-cost** path to any unexplored state is stored
  - States are cached **after** expansion (i. e. after removing path from list)
- This variant is known as *minimal-cost search* (MCS) or *uniform cost search* (UCS)

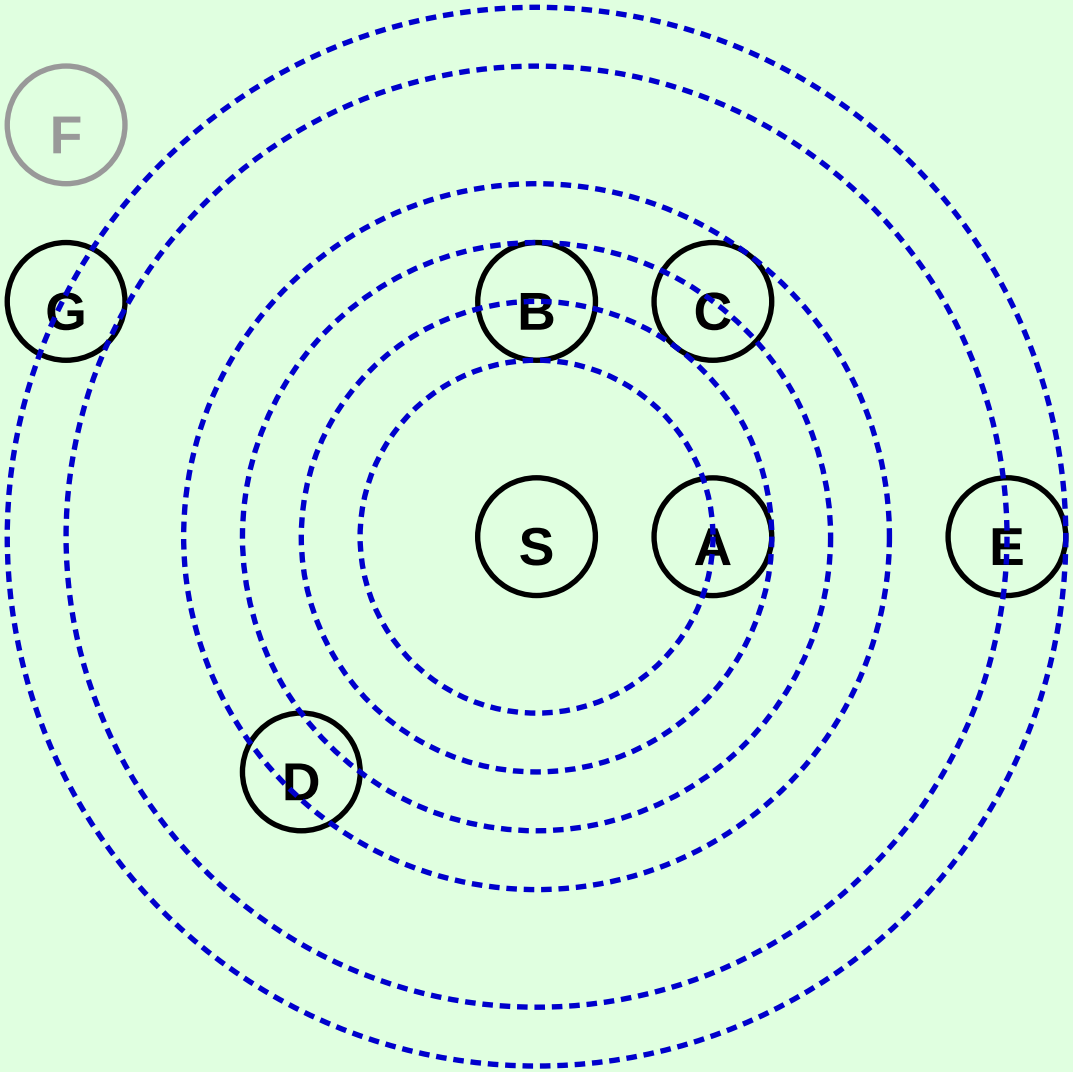
Minimal-cost search

```
def costsearch (model, depth):
    (start, goal, successors, cost) = model
    if goal(start): return [start]
    frontier = [ [start] ]
    explored = set([ start ])
    while frontier:
        path = get_path(frontier)
        state = path[-1]
        if goal(state): return path
        explored.add(state)
        if path_len(path) >= depth: continue
        for (action, new_state) in successors(state):
            if new_state not in explored:
                new_path = path + [action, new_state]
                curr = existing_path_to(new_state)
                if curr:
                    if cost(new_path) < cost(curr):
                        frontier.remove(curr)
                    else: continue
                frontier.insert(new_path, cost)
    return []
```

Minimal-cost search: evaluation

- Behaves similarly to BFS
- Complete and optimal algorithm
  - Provided *positive* action costs
- Time and memory complexity is $O(b^{1+C/c})$
  - C/c – ratio of solution path cost and typical step cost

Minimal-cost search „contour“



Bidirectional search

- Searching from both initial ***and*** goal states
  - Requires invertible or symmetrical successor function
  - BFS, IDS or UCS is used
  - Goal check is replaced by *frontier collision*
- Complete and optimal algorithm
- Algorithmic complexity – $O(b^{d/2})$
  - Still exponential, but allows searching in larger spaces

Informed search

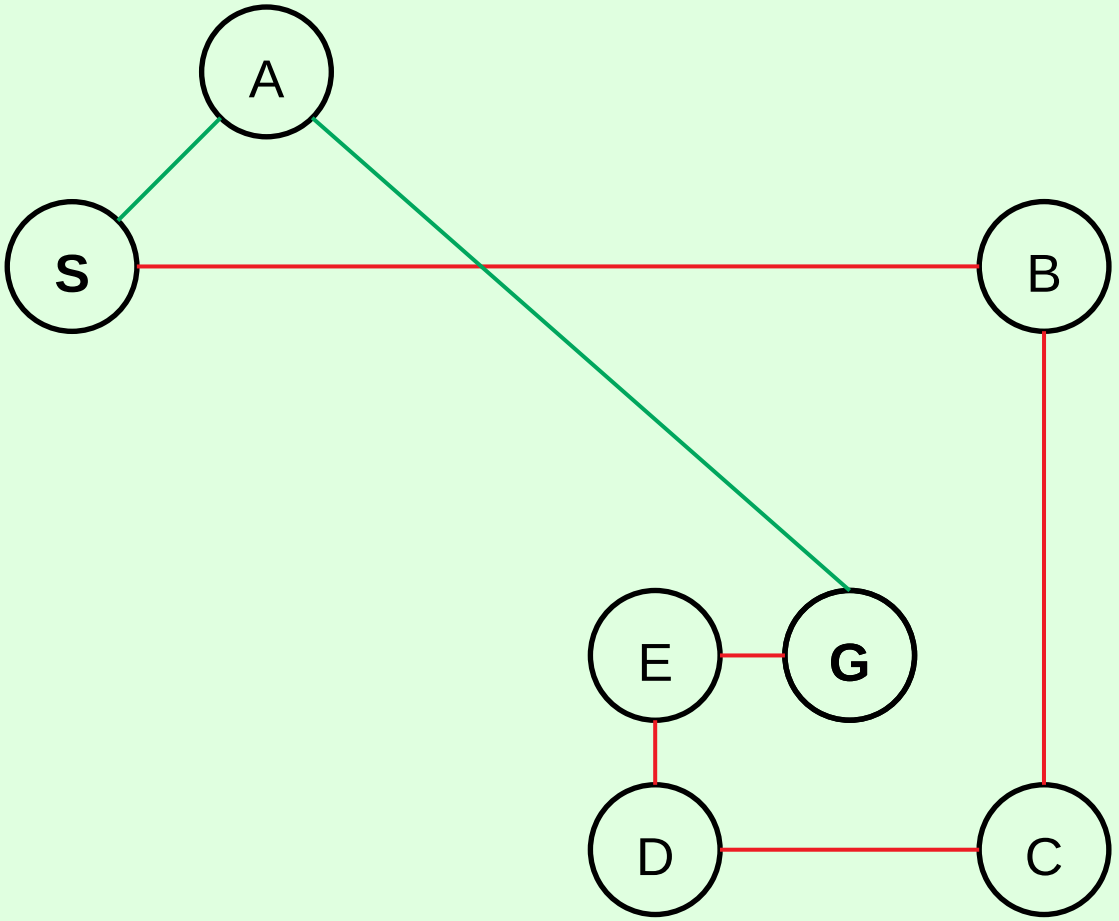
- Information available for the agent:
  - Initial state
  - Traversal scheme
  - Goal check
  - **State valuation („goodness“) function**

- The most popular – heuristic *function h(s)*
  - *Estimated* cost of path **left** until goal is reached
- *h(s)* must be *optimistic*
  - Estimated cost must be *less or equal* to the actual cost
  - Valid heuristic is also called *admissible*

Best-first search strategy

- Path with lowest heuristic *h(s)* is expanded first
  - Also known as *greedy search*
  - Now search has some direction towards (supposed) goal
  - Algorithm is identical to MCS, just using different cost function
- Algorithm is ***neither*** complete ***nor*** optimal
  - The heuristic-pointed direction is overvalued compared to all others
  - Has resemblance to DFS (also belonging to a family of *greedy* methods)
- Time and space complexity – $O(b^m)$
  - Effectiveness is mostly determined by *h(s)* selection
  - For infinite *m*, notation b^{d*} („*effective d*“) is probably more sensible

Best-first, worst-case

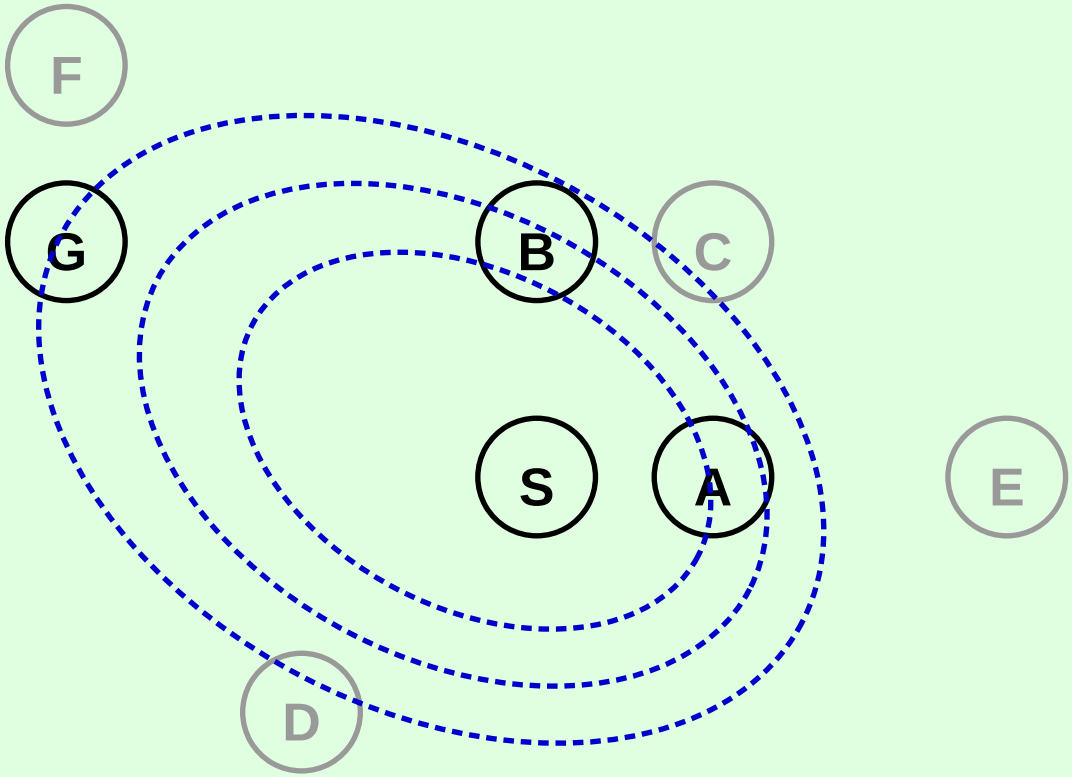


A search*

- Path with lowest **sum** $c(p)+h(s)$ is selected
  - Identical to MCS in implementation

- Complete and optimal algorithm
- Time and space complexity – $O(b^d)$
  - Use of heuristics can be seen as finding „effective b“, b^*

A search contour*



*Memory-bounded A**

- MA*, SMA* – A* search variants
  - Frontier size is limited; paths are dropped upon reaching the limit
  - Highest-cost and earliest-found paths are dropped first
  - Costs of dropped paths are being retained
  - If frontier becomes empty (or small), dropped paths are regenerated
- Algorithm is complete, if solution can fit in memory
- …and optimal, if *optimal* solution can be stored
- IDA* and RbFS use even stricter memory bounds
  - As always, lower memory costs mean more time (computation) costs

Choice of suitable heuristic

- A can slide to B:
  - If A and B are adjacent
  - If B is empty
- Solution – up to 80 steps
- Around 10^{13} possible states
- How to choose *h(s)*?



Determining heuristics

- A can slide to B:
  - If A and B are adjacent
  - ~~If B is empty~~
- h_1 – Manhattan distances

- A can slide to B:
  - ~~If A and B are adjacent~~
  - ~~If B is empty~~
- h_2 – no. of out-of-place tiles

- h_1 and h_2 are exact solutions of *simplified* (*relaxed*) problem
  - Such functions are (by design) admissible
  - Solving time of relaxed problem(s) can be negligible
- $h_1 \geq h_2$, so h_1 is better (closer to truth) than h_2
- However, there is no need to even choose: $h_{opt} = max(h_1, h_2)$

Local search

- *Non-systematic* search
  - Only current graph node and closest neighbors are examined
  - Search paths are usually not stored
  - ***In***complete, ***non***-optimal
- Suitable for infinite/ even uncountable state spaces
  - Memory is bounded by algorithm or hardware
- „Close enough“ solutions
  - Numerical checks are used instead of strict equality
- I’m describing something you’ve surely already met
  - So-called *optimization* methods

Local search methods

- Downhill search
  - To a closest local extremum
  - Can be improved by performing search from different starting points
- Simulated annealing
  - Traversal criteria become more strict while performing search
  - Allows finding and selecting from multiple local extrema
- Local beam search
  - Parallel local search in neighboring areas of state space, with either pooling from or information exchange between individual threads
- Continuous-space search
  - Gradient-based methods (using first derivatives or conjugate directions)
  - Newton method(s) (using second derivatives)